# Тестовое задание
Реализация Attention слоя и оптимизация матричного умножения на C++

## Цель задания

Реализовать **Scaled Dot-Product Attention** (из статьи “Attention Is All You Need” https://arxiv.org/pdf/1706.03762) на C++ с несколькими вариантами матричного умножения и проанализировать произво- дительность различных подходов.

## 1 Формулировка задачи

Реализовать функцию внимания:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Где:

- **Q** (Queries) — тензор размером [batch_size, seq_len$_q$, $d_k$]
- **K** (Keys) — тензор размером [batch_size, seq_len$_k$, $d_k$]
- **V** (Values) — тензор размером [batch_size, seq_len$_k$, $d_v$]
- $d_k$ — размерность ключей/запросов
- $d_v$ — размерность значений

**Выход:** тензор размером [batch_size, seq_len$_q$, $d_v$]

## 2 Требования к реализации

### 2.1 Базовые требования

- **Язык:** C++17/20
- **Тип данных:** float
- **Хранение тензоров:** на ваше усмотрение
- **Параметры:** передаются как аргументы функции

## 2.2 Функция Attention

```
/**
 *                    Scaled Dot-Product Attention
 *
 * @param Q - [batch, seq_q, d_k]
 * @param K - [batch, seq_k, d_k]
 * @param V - [batch, seq_k, d_v]
 * @return [batch, seq_q, d_v]
 */
Tensor attention(
    const Tensor& Q,
    const Tensor& K,
    const Tensor& V
);
```

Листинг 1: Заголовок функции attention

## 2.3 Реализации матричного умножения

Необходимо реализовать **три различных подхода** к умножению матриц, которые будут использоваться внутри Attention:

**Вариант A: Наивное тройное умножение**

**Вариант B: Кэш-оптимизированное умножение**

**Вариант C(бонусный): SIMD-векторизация**

## 2.4 Структура Attention с разными matmul

Реализация Attention должна быть параметризована типом умножения:

```
enum MatMulType {
    NAIVE,
    CACHE_OPTIMIZED,
    SIMD
};

Tensor attention_with_matmul(
    const Tensor& Q,
    const Tensor& K,
    const Tensor& V,
    MatMulType matmul_type
);

```

Листинг 2: Параметризованная функция attention

# 3 Дополнительная задача

## 3.1 FlashAttention-подобная оптимизация

Объединить softmax и второе умножение в один проход (online softmax), чтобы не материализовать полную матрицу внимания.

# 4 Ожидаемый результат

На проверку предоставляется:

1. ReadME с подробным описанием сборки/запуска, реализованных методов и результатами измерений
2. Gitlab со всеми реализациями

**Успешной реализации!**